

Module - Plateformes de Développement

Spring

Support de cours

Par :

Dr. Ing. Faouzi JAIDI

faouzi.jaidi@gmail.com

2021-2022

Plan

- 1. Introduction
- 2. Maven
- 3. Logs : Log4J
- 4. Tests: JUnit
- 5. Tests & Qualité : Sonar
- 6. IOC & ID
- 7. Spring Boot
- 8. Spring Data JPA
- 9. Spring MVC REST
- 10. JSF
- 11. AOP
- 12. Projet
- 13. Spring Security
- 14. Extensions

Spring

***IoC** - Inversion de Contrôle & **ID** - Injection de Dépendance*



IoC & ID

□ IoC

- IoC : Inversion of Control / Inversion de Contrôle.
- Un **principe d'architecture** (*architectural pattern* / *patron d'architecture*) conduisant à **inverser** le flux de contrôle par rapport au développement traditionnel.
 - *L'exécution de l'application n'est plus sous le **contrôle direct** de l'application elle-même mais du Framework sous-jacent.*
- Comment l'application est contrôlée par le Framework Spring?
 - *L'Injection de Dépendance (ID).*

IoC & ID

□ ID

- ID : Injection de Dépendance.
- L'ID est la forme principale de l'IoC. Elle implémente le principe de l'IoC.
- Un mécanisme permettant de diminuer le couplage entre deux ou plusieurs objets.
- Une méthode pour instancier des objets et créer les dépendances nécessaires entre eux:
 - Réduit le code relatif aux dépendances;
 - Permet une meilleure lisibilité du code;
 - Minimise / optimise le temps de développement.

IoC & ID

□ Conteneur IoC

- L'inversion de contrôle implique l'existence d'un composant supplémentaire capable d'instancier les objets et d'injecter des objets dans d'autres.
 - *Conteneur IoC.*
- Le Conteneur IoC « *IoC Container* » accueille un ensemble d'objets dont il a la responsabilité de gérer le cycle de vie.
- Le conteneur IoC est constitué d'un ou de plusieurs contextes d'application.
- Le Spring Framework est avant tout un conteneur IoC.

□ Contexte d'application

- Un contexte d'application contient la définition des objets que le conteneur doit créer ainsi que leurs interdépendances.
- L'API du Spring Framework définit l'interface `ApplicationContext` et plusieurs implémentations de cette interface:

- `ClassPathXmlApplicationContext` : définir un contexte d'application à l'aide d'un fichier XML.

```
private ApplicationContext context;  
...  
context = new ClassPathXmlApplicationContext("demo-beans.xml");
```

- `AnnotationConfigApplicationContext` : définir un contexte d'application à l'aide des annotations.

```
private ApplicationContext context;  
...  
context = new AnnotationConfigApplicationContext(BeansConfiguration.class);
```

- ...

IoC & ID

□ Bean Spring

Les Java Beans sont des composants logiciels réutilisables manipulables visuellement dans un outil de conception.

- Un bean est un composant Java spécifique avec :
 - ***class***: fully qualified java class name,
 - ***id*** : the unique identifier for this bean,
 - ***configuration*** : (scope, init-method, etc.),
 - ***property***: arguments to pass to the bean setters at creation time.
- Un bean spring est un bean géré par le framework Spring.
- Un bean est défini avec une configuration XML, Java ou par annotation pour que Spring puisse le gérer.

□ Notion de portée (scope)

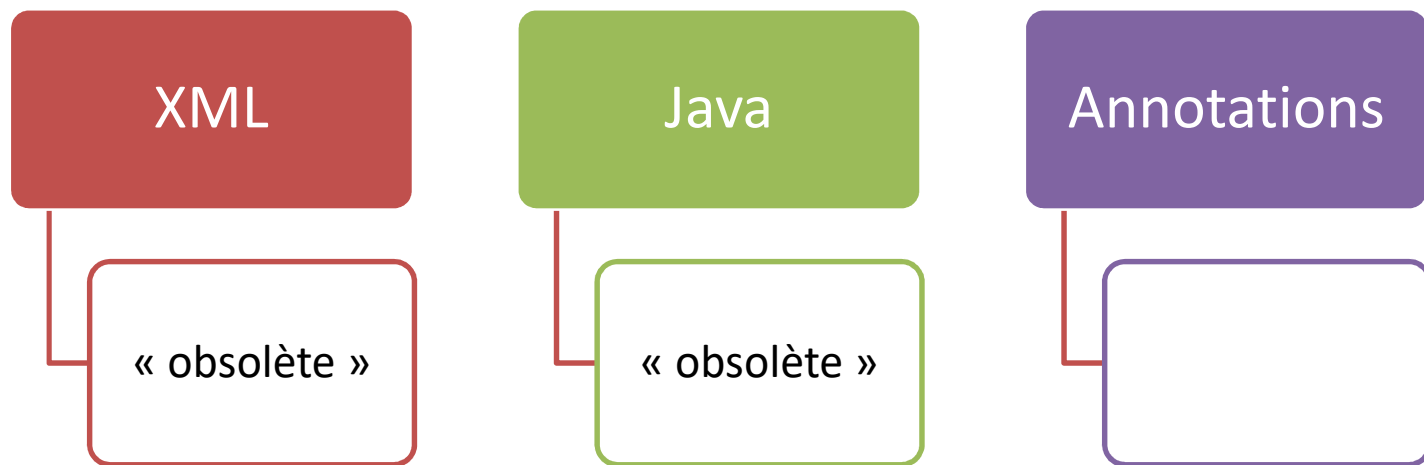
Les beans ajoutés dans un contexte d'application ont une portée (scope).

- Différents types de Bean (scope): :
 - **singleton**: [scope par défaut] une seule instance du bean est créée pour chaque conteneur IoC Spring et référencée à chaque invocation.
 - **prototype**: une nouvelle instance est créée à chaque appel du bean.
 - **request**: [Contexte Web uniquement] une nouvelle instance est créée pour chaque requête http / https.
 - **session**: une instance du bean par session http.
 - **global-session** : une instance du bean par session globale (valable pour les portlets: ensemble d'applications déployées sur un même conteneur web).

IoC & ID

□ Configuration

Trois manières différentes pour configurer l'injection de dépendances et la gestion des beans Spring:



IoC & ID

o XML

```
<bean id="userControl" class="tn.enicarthage.control.UserControlImpl">  
  <property name="userService" ref="userService" />  
</bean>
```

```
public class UserControlImpl {  
    IUserService userService;  
    public IUserService getUserService() {  
        return userService;  
    }  
    public void setUserService(IUserService userService) {  
        this.userService = userService;  
    }  
}
```

IoC & ID

O Java

```
@Configuration
public class BeansConfiguration {

    @Bean(name="userControl")
    public UserControlImpl createUserControl()
    {
        UserControlImpl uc = new UserControlImpl();
        uc.setUserService(createUserService());
        return uc;
    }

    @Bean(name="userService")
    public UserServiceImpl createUserService()
    {
        UserServiceImpl us = new UserServiceImpl();
        us.setUserDAO(createUserDAO());
        return us;
    }
}
```

IoC & ID

o Annotations

```
@Configuration
@ComponentScan(basePackages = {"tn.enicarthage.control",
"tn.enicarthage.service", "tn.enicarthage.dao"})
public class BeansConfiguration {
}
```

```
@Controller
public class UserControlImpl implements IUserControl {

@Autowired
IUserService userService;
```

```
@Service
public class UserServiceImpl implements IUserService {

@Autowired
@Qualifier("ud")
IUserDAO userDAO;
```

IoC & ID

o Quelques Annotations

- **@Autowired** : permet d'activer l'injection automatique de dépendance.
- **@Primary**: permet d'indiquer le *bean* qui devra être sélectionné en priorité en cas d'ambiguïté.
- **@Qualifier**: permet de qualifier, c'est-à-dire de préciser le nom du *bean* à injecter.
- **@Value**: est utilisable sur un attribut ou un paramètre pour un type primitif ou une chaîne de caractères. Elle donne la valeur par défaut à injecter.
- ...

IoC & ID

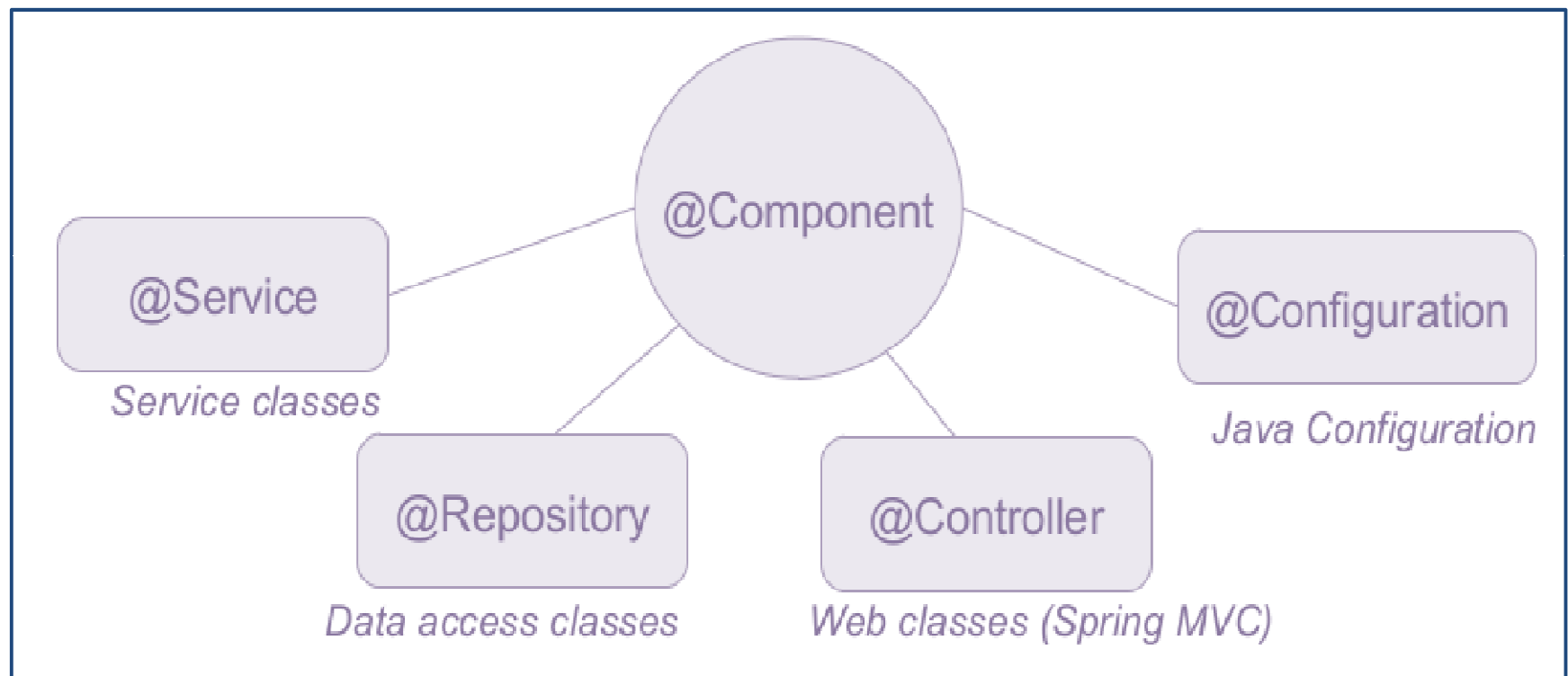
○ Comparaison

	<i>Java</i>	<i>Annotations</i>	<i>XML</i>
<i>Avantages</i>	- Configuration centralisée - Typage fort	- Développement rapide - La classe contient sa configuration	Legacy / Obsolète
<i>Inconvénients</i>	- Nécessite plus de code par rapport aux annotations	- Configuration éparpillée. - Configuration et code dans un même endroit.	

➤ *Il est possible de combiner les différents types de configuration !*

IoC & ID

□ Stéréotypes des Beans





Spring

TP

